4 ARRAYS AND POINTERS

4.1 Array

- An array is defined as the collection of similar type of data items stored at contiguous memory locations.
- Arrays are the derived data type in C programming language which can store the primitive type of data such as int, char, double etc.
- It has the capability to store the collection of derived data types such as pointer, structure etc.
- Each element of an array is of same data type and carries the same size example int = 4 byte.
- Elements of the array are stored at contiguous memory locations, meaning, the first element is stored at the smallest memory location.
- Elements of the array can be randomly accessed since we can calculate the address of each element of array with the given base address and size of the data element.

4.1.1 Advantages of C Array

1) Code Optimization: Less code to access the data.
2) Ease of traversing: By using the for loop, we can retrieve the elements of an array easily.
3) Ease of sorting: To sort the elements of the array, we need a few lines of code only.
4) Random Access: We can access any element randomly using the array.

4.1.2 Disadvantages of Array

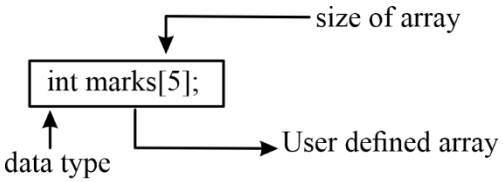
- **Fixed Size**: **1)** Whatever size, we define at the time of declaration of the array, we can't exceed the limit.
  2) It does not grow the size dynamically like linked list.

Declaration of C Array

Syntax:

```
data type array name [array_size]
```

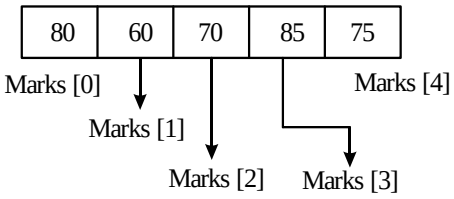
Example:



Initialization of C array:

We can initialize each element of the array by using the index. Suppose array int marks [5]. Then

```
marks [0] = 80
marks [1] = 60;
marks [2] = 70;
marks [3] = 85;
marks [4] = 75;
```



C Array Example:

```
# include <stdio.h>
int main ()
{
int i = 0;
int marks [5];
marks [0] = 80;
marks [1] = 60;
marks [2] = 70;
marks [3] = 85;
marks [4] = 75;
for(i = 0; i<5; i++)
{
    printf("%d\n",marks[i]);
}
return 0;
}
```

4.1.3 Array: Declaration with Initialization

- We can initialize the C array at the time of declaration.
  Example: int marks [5] = {20, 30, 40, 50, 60};
- In such case, there is no requirement to define the size.
  Example: int marks [ ] = {20, 30, 40, 50, 60}

Example:

```
# include<stdio.h>
int main ()
{
int i = 0;
int marks [5] =  {20, 30, 40, 50, 60}
for(i = 0; i < 5; i++)
    {
        printf("%d\n",marks[i]);
    }
return 0;
}
```

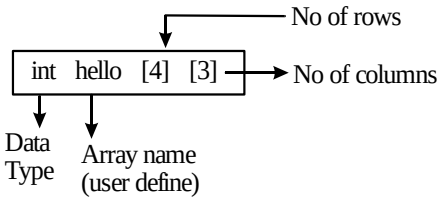
4.1.4 Two-Dimensional Array

- The two-dimensional array can be defined as an array of arrays.
- The 2D array is organized as matrices which can be represented as the collection of row and columns.

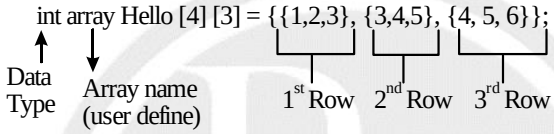
4.1.5 Declaration of two-dimensional array in C syntax

```
data_type array_name [rows] [columns]
```

Example:



Initialization of 2D Array



Example: Program of Two-Dimensional Array

```
# include<stdio.h>
int main()
{
    int i = 0, j = 0;
    int arrayHello[4][3] = {{1, 2, 3}, {2, 3, 4}, {4, 5, 6}};
    for(i = 0; i < 3; i++)
    {
    for(j = 0; j < 3; j++)
        {
            printf("arrayHello[%d][%d]=%d\n",i,j,arrayHello[i][j])
        }
    }
    return 0;
}
```

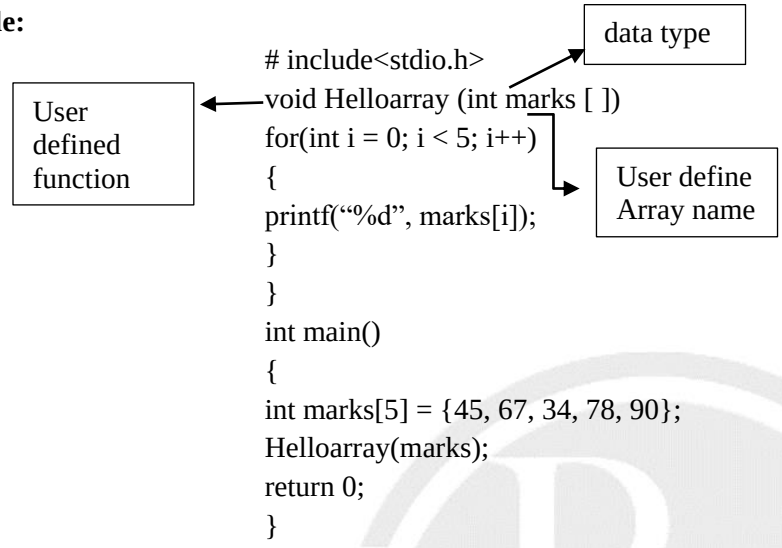
Output:

```
arrayHello[0][0] = 1
arrayHello[0][1] = 2
arrayHello[0][2] = 3
arrayHello[1][0] = 2
arrayHello[1][1] = 3
arrayHello[1][2] = 4
arrayHello[2][0] = 3
arrayHello[2][1] = 4
arrayHello[2][2] = 5
arrayHello[3][0] = 4
```

arrayHello[3][1] = 5
arrayHello[3][2] = 6

4.1.6 Passing array to a function

Example:




```
# include<stdio.h>
void Helloarray (int marks [ ])
for(int i = 0; i < 5; i++)
{
printf("%d", marks[i]);
}
}
int main()
{
int marks[5] = {45, 67, 34, 78, 90};
Helloarray(marks);
return 0;
}
```

Note: void Helloarray(int marks[ ])

Without return type function

4.1.7 Passing Array to a Functions as a pointer

Now, we will see how to pass an array to a function as a pointer.

```
# include<stdio.h>
void helloarray(char * marks)
{
    printf("Elements of array are:");
    for(int i = 0; i < 5; i++)
    {
    printf("%c", marks[i])
    }
    int main()
    {
    char marks[5] = {'A', 'B', 'C', 'D', 'E'};
    helloarray(marks);
    return 0;
    }
}
```

Note:
Whenever you pass an array, it is always call by reference. In previous 2 programs, we passed an address & formal argument in both the cases is a pointer internally.

How to return an Array from a function

```
# include<stdio.h>
int *fun()
{
int marks [5];
print f (“Enter the element in an array”);
for (int i = 1; i < 5; i++)
{
scanf(“%d”,& marks[i]);
}
return marks;
}
int main()
{
int *n;
n = fun();
printf(“\n Elements of array are:”);
for(int i = 0; i < 5; i++)
{
printf(“%d”,n[i]);
}
return 0;
}
```

Note:
fun() function returns a variable ‘marks’.
It returns a local variable, but it is an illegal memory location to be returned, which is allocated with a function in the stack. Since the program control comes back to the main () function, and all the variables in the stack are fraud.
Therefore, we can say that this program is returning memory location, which is already de-allocated, so the output of the program is a **segmentation fault.**

4.1.8 Array declaration and initialization

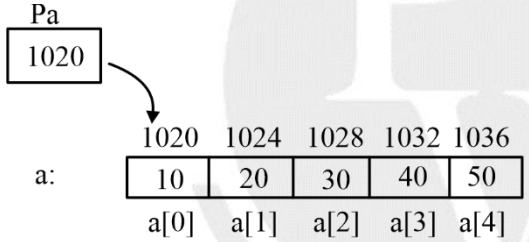
1. int A[ ] = {10, 20, 30}: valid
2. int A[3] = {10, 20, 30}: valid
3. int A[ ] ; invalid
4. int A[3] ; valid
5. int A[2][3]; valid
6. int A[ ] [3]; = {10, 20, 30}: invalid
7. int A[2] [3]; = {1, 2, 3, 4, 5, 6}: valid
8. int A[ ] [3]; = {1, 2, 3, 4, 5, 6}: valid
9. int A[2] [3] [2]; invalid
10. int A[ ] [3] [2]; invalid
11. int A[ ] [2] [ ]; invalid
12. int A[ ] [ ] [2]; invalid
13. int A[2] [3] [2]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; valid

14. int A[ ] [3] [2]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; valid
15. int A[2] [ ] [2]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; invalid
16. int A[2] [3] [ ]; = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12,}; invalid

Note:
- while declaring an array, it is mandatory to provide the size of each dimension. (3), (6), (10), (11), (12) are not providing sizes of dimensions.
- While initializing an array, you have flexibility to omit only first dimension i.e., you are allowed to other dimension (1), (8),(14) are following this rule while (15), (16) are not following this rule.

4.1.9 Pointers

- In C, there is a strong relationship between arrays and pointers. An operation that can be achieved by arrays subscripting also be done with pointes.
- Consider the declaration
  int a[5] = {10, 20, 30, 40, 50};
  int* Pa;
  Pa = &a[0];
- Pointers are special variable that can hold address of other variables
  Syntax: data type * Identifier
  int *Pa: Pa is a pointer to integer variable
  i.e., Pa can hold address of integer variable



Pa contains address of a[0]

- Two operators are important to understand for understanding pointers: &, *
  i. &: address of operator
  ii. *: value at operator
- Pa is equivalent address 1020
- Pa is equivalent to value at (Memory location 1020)
  i,e *Pa is same as 10
- Pa is pointing to same element of array, then by definition Pa +1 points to next element, Pa + i points to elements after i elements before Pa.
  i.e, In our example Pa points to a[0]
  So, Pa+1 will point [1]
  Pa+2 will point to a[2]
  i,e
  Pa+i is address of a[i]
  $\Rightarrow$ Pa + i ≡ & a[i]
  $\Rightarrow$ *(Pa+i) ≡ *& a[i] ≡ a [i]
- Array name always represent address of the very first element
  i.e., Pa = &a[0];
  and

Pa = a ;

both are same.

- a[i] ≡ *(a+i) ≡ *(i + a) ≡ i [a]
  All are same and can be used interchangeably in program except in declaration.
- In declaration, we can not write
  int 3[a]; instead of int a[3]
- Array name represent a constant address.
  int a[10];

i. a++, ++a, – –a , a – – are all invalid as we cannot apply increment/decrement operator on constants:

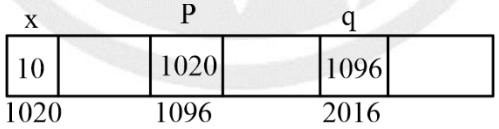
ii. Array name cannot be Lvalue of on assignment statement.
i.e.
array_name = any expression/address/value is invalid

- On the other hand, pointer being a variable, the following operations are valid.

```
int a[5]
int = *Pa;
Pa = a : valid
Pa+ + ;
Pa – –:
+ + Pa :
– – Pa :
```
} *all are valid*

- Multi-level pointer

```
int x      :   x can store value
int *P     :   p can store address of integer variable
int **q    :   q can store address of pointer variable
x = 10     :   valid
p = &x     :   valid
q = &p     :   valid
```



p ≡ 1020 i, e address of x
*P ≡ value at (memory location 1020)
*p ≡ 10
q ≡ 1096 i,e address of variable p
*q ≡ value at (memory location 1096)
≡ 1020 which is again an address
*q ≡ value at (memory address 1020)
**q ≡ 10

1. **Dangling pointer:** The pointer pointing to a deallocated memory block is known as dangling pointer.
   This situation raises an error known as dangling pointer problem. Dangling pointer occurs when a pointer pointing to a variable goes out of scope or when a variables memory gets deallocated.
   Consider the code.
   int *f( ) {

```
int a = 10; // a is a local variable and goes out of scope after execution of f( )
return &a :
}
int main () {
int * ptr = f (); //Ptr points to something which is not valid
print ("/d"*,ptr);
return 0;
}
```

- To overcome this problem, just make variable a as static when x become static it has scope throughout the program.

2. **Uninitialized pointer:** An uninitialized pointer also known as wild pointer
   A pointer which has not been initialized to anything can be dangerous

- The value saved in an uninitialized pointer could be randomly pointing anywhere in memory
- int *p

- int *p

- Storing a value using an uninitialized pointer has the potential to overwrite anything in your program including your program itself.
- System may crash.
- Always initialize with NULL.

3. **NULL pointer:** It is a pointer which is pointing to nothing It is a specially designed pointer that stores a defined value but not a valid address of any element or object.
   NULL pointer does not hold valid address and that is why if you try to dereference it, you will get an error

```
int *p = NULL:
printf ("%d",*p) :        //NULL pointer dereferencing
```

4. **void pointer**: void pointer is a pointer that points to some location in memory but is does not have any specific type.
   It can point to any type of data and any pointer type is convertible to a void pointer.
   Its declaration:

```
void *ptr :
```

   is saying that ptr is a pointer that can hold an address
   cannot be dereferenced directly
   i.e.,

```
void *ptr
int x = 10:
```

```
ptr = &x :
printf ("%d", *ptr);        //invalid
```

you need to typecast the void pointer before dereferencing.

i.e.,

```
void *ptr
int x = 10;
ptr = &x;
printf("%d",*(int*)ptr);
                 └→typecasting
```

Here, (int*)ptr does type costing of void

(int) ptr dereferences the typecasted void pointer variable

- Pointer arithmetic in not possible on void pointer.
  i,e……. ptr+ +, + + ptr, – –ptr, ptr – –, ptr +1, ptr -1 is not allowed an void pointer.

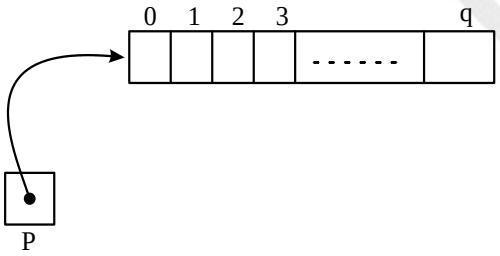
- An uninitialized pointer holds a garbage value while a NULL pointer holds a defined value but not a valid address.

4.1.10 Memory leakage problem

- Whenever a pragrammer allocated memory dynamically then it is the reponsibility of the programmer to free that memory after usage.
- Memory leakage problem occurs when a programmer allocates memory dynamically but does not de-allocate it after using it.
- It reduces the performance of the computer by reducing the amount of availbale memory.

4.1.11 Understanding Declartion

1. int *(P[10]) : P is an array of 10 pointer to integer
2. int (*P)[10] : P is a pointer to an array of 10 integers



3. int (*P)() : P is pointer to a function that takes no argument and returns an integer.
4. int (*P) (int, int) : P is a pointer to a function that takes 2 integer arguments and returns an integer.
5. int *P (char*) : P is a pointer to a function that takes a pointer to character argument and return a pointer to integer.

4.1.12 Pointer to Functions / Function Pointer

- Just like normal pointers we can have pointers to functions.

```
int(*p)(int, int)
```

P is a pointer to a function that take 2 integer arguments and returns an integer value.

- **Declaration of function pointer:** declaring a pointer to function is almost same as declaring the fuction except that in function pointer are prefix is with an asterik * symbol.
  For example: if the function is

  int add(int, int)

  Declaration of a fuction pointer for add() fuction is :

  int (*ptr)(int, int);
- How to call a function through function pointer : In order to all a function using function pointer, first we need to assign the address of function code to the pointer

Ex 1:

```
int add (int, int):
void main () {
        int (*ptr) (int, int);
        ptr = &add;       // ptr is a pointer to add() function
printf ("The sum of 10 and 20 is", (*ptr )(10, 20)); //calling add()
}
int add (int, int);
```

Ex 2:

```
void main() {
        int (*ptr) (int, int);
        ptr =  add ; // same as ptr = & add
printf (" The  sum of  10 and 20 is", (*ptr) (10, 20));
```

- A function can be called using following 4 ways using pointer to fuction.

int add (int int);
void main() {
int (*ptr) (int, int);
ptr = &add;
printf ("%d", (*ptr)(10, 20));
}
	int add (int int);
	void main() {
	int (*ptr) (int, int):
	ptr = add;
	printf ("%d", (*ptr) (10, 20));
	}

int add (int, int) ;
void main() {
int(*ptr) (int, int);
prt= &add;
printf ("%d", (*ptr)(10, 20));
}
	int add (int, int);
	void main() {
	int (*ptr) (int, int);
	prt= add;
	printf ("%d", (*ptr) (10, 20));
	}

- Using function poitnter we are able to pass a function as an argurment to other function and can also be returned from function.
  Array of function pointer
  int (*ptr[3])(int, int)
  Ptr is an array of 3 pointers to a function that takes 2 arguments and returns an integer.

❐❐❐